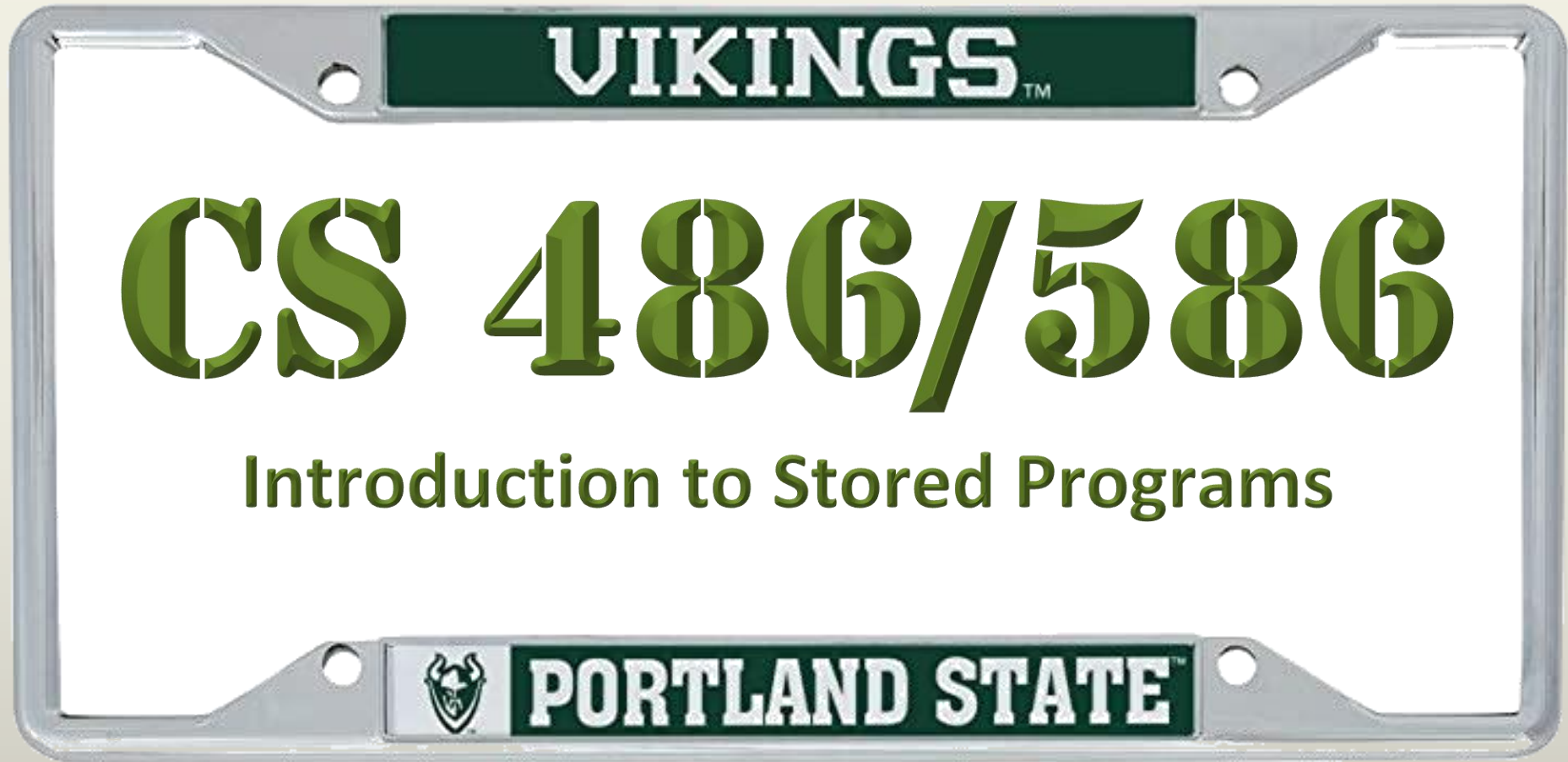




Heavily based on material from:
Murach's MySQL (3rd Edition), by Joel Murach

Three types of stored programs

1. **Stored procedure**

- Can be **called from an application that has access to the database.**

2. **Stored function**

- Can be called from a SQL statement. A stored function works **much like the functions provided by PostgreSQL** (sum, count, ...).

3. **Trigger**

- Is executed in response to an **INSERT, UPDATE, or DELETE** statement on a specified table.

SQL statements for controlling the flow of execution

`IF...ELSEIF...ELSE`

`CASE...WHEN...ELSE`

`WHILE...DO...LOOP`

`DECLARE CURSOR FOR`

A SQL statement used within stored programs

`SELECT`

A stored procedure that doesn't do anything

```
CREATE OR REPLACE PROCEDURE sproc.test0()  
LANGUAGE plpgsql  
AS  
$$  
BEGIN  
    -- perform 'This is a test.' AS message;  
    RAISE NOTICE 'message: %', 'This is a test';  
END;  
$$  
  
call sproc.test0();
```

This executes the query and discards the result.

The syntax for declaring a variable

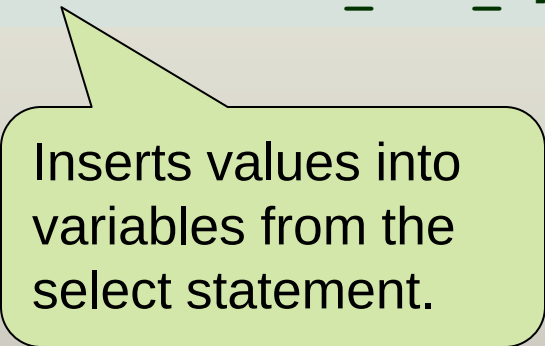
```
DECLARE variable_name data_type [DEFAULT literal_value];
```

The syntax for setting a variable to a literal value or an expression

```
variable_name := {literal_value|expression};
```

The syntax for setting a variable to a selected value

```
SELECT column_1[, column_2]...  
INTO variable_name_1[, variable_name_2]...
```



Inserts values into variables from the select statement.

A stored procedure that uses variables

```
CREATE OR REPLACE PROCEDURE sproc.test1()  
LANGUAGE plpgsql  
AS  
$$  
DECLARE  
    max_invoice_total    real = 4;  
    min_invoice_total    real = 3;  
    percent_difference    real = 2;  
    count_invoice_id     INT = 1;  
    vendor_id_var        INT = 95;  
begin  
    percent_difference :=  
        (max_invoice_total - min_invoice_total) /  
        min_invoice_total * 100;  
  
    RAISE NOTICE 'message: %', percent_difference;  
end;  
$$  
call sproc.test1();
```

A stored procedure that uses variables

```
CREATE OR REPLACE PROCEDURE sproc.test2()  
LANGUAGE plpgsql  
AS  
$$  
declare  
    sp_value int = 1;  
    sp_name text = 'Jesse';  
BEGIN  
    sp_name := 'Jesse';  
    sp_name = (select name  
               from transaction_tests.test  
               where id = 1);  
    select value, name  
    from transaction_tests.test  
    into sp_value, sp_name  
    where id = 1;  
    RAISE NOTICE 'message: %', sp_name;  
END;  
$$
```

The syntax of the IF statement

```
IF boolean_expression THEN
    statement_1;
    [statement_2;]...
[ELSEIF boolean_expression THEN
    statement_1;
    [statement_2;]...]...
[ELSE
    statement_1;
    [statement_2;]...]
END IF;
```

Note

- You can also code parentheses around the Boolean expressions in an IF statement. The parenthesis are optional.

I like parenthesis.


```
CREATE or replace PROCEDURE sproc.test3()  
language plpgsql  
as$$  
DECLARE  
    first_sale_date DATE;  
    message text;  
BEGIN  
    SELECT MIN(date_of_sale)  
    INTO first_sale_date  
    FROM books3.book_sales  
    WHERE quantity > 5000;  
  
    IF first_sale_date < NOW() THEN  
        SELECT 'Outstanding invoices are overdue!' into message;  
    ELSEIF first_sale_date = NOW() THEN  
        SELECT 'Outstanding invoices are due today!' into message;  
    ELSE  
        SELECT 'No invoices are overdue.' into message;  
    END IF;  
    RAISE NOTICE 'message: %', message;  
END;  
$$
```

The syntax of the simple CASE statement

```
CASE expression
  WHEN expression_value_1 THEN
    statement_1;
    [statement_2;]...
  [WHEN expression_value_2 THEN
    statement_1;
    [statement_2;]...]...
  [ELSE
    statement_1;
    [statement_2;]...]
END CASE;
```



constants

```
CREATE or replace PROCEDURE sproc.test4()  
language plpgsql  
as  
$$  
DECLARE  
    edition_id INT;  
    message text;  
BEGIN  
    SELECT id INTO edition_id  
    FROM books3.editions WHERE edition = 'English' and book_id = 1;  
    select  
        CASE edition_id  
            WHEN 1 THEN 'English'  
            WHEN 2 THEN 'Latin'  
            WHEN 3 THEN 'Greek'  
            ELSE 'Hindi'  
        END msg into message;  
    RAISE NOTICE 'message: %', message;  
END $$
```

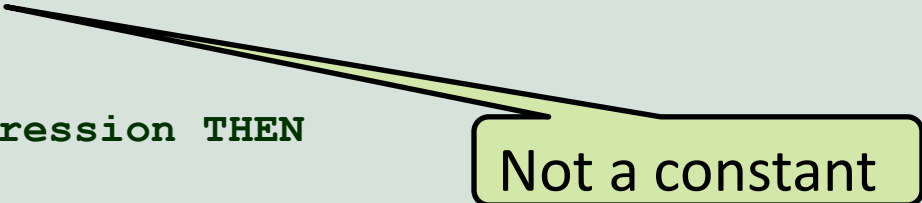
The syntax of a searched CASE statement

```
CASE
  WHEN boolean_expression THEN
    statement_1

  [WHEN boolean_expression THEN
    statement_1]

  [ELSE
    statement_1]

END;
```



Not a constant

The syntax of the WHILE loop

```
WHILE boolean_expression LOOP  
    statement_1;  
    [statement_2;] ...  
END LOOP;
```

- You can use the **CONTINUE** statement to go to the beginning of a loop. This is like the **continue** statement in C.
- You can use the **EXIT** the innermost loop is terminated and the statement following `END LOOP` is executed next. This is like the **break** statement in C.

A stored procedure that uses a WHILE loop

```
CREATE or replace PROCEDURE sproc.test5()  
language plpgsql  
as  
$$  
DECLARE  
    i INT DEFAULT 1;  
    s VARCHAR(400) DEFAULT '';  
BEGIN  
    WHILE i < 5 LOOP  
        s = CONCAT(s, 'i=', i, ' | ');  
        i = i + 1;  
    END LOOP;  
    raise notice 'message: %', s;  
  
END;  
$$
```

	message
▶	i=1 i=2 i=3

The output for this code

A simple loop

```
do $$
declare
    counter int = 0;
begin
    loop
        counter = counter + 1;
        -- exit the loop if counter > 10
        exit when counter > 10;
        -- skip the current if counter is an even number
        continue when mod(counter,2) = 0;
        -- print out the counter
        raise notice 'counter = %', counter;
    end loop;
end; $$
```

- You can use the **CONTINUE** statement to go to the beginning of a loop. This is like the **continue** statement in C.
- You can use the **EXIT** the innermost loop is terminated and the statement following **END LOOP** is executed next. This is like the **break** statement in C.

```
CREATE or replace PROCEDURE sproc.test7()  
language plpgsql  
as $$  
declare  
    counter int = 0;  
begin  
    loop  
        counter = counter + 1;  
        -- exit the loop if counter > 10  
        exit when counter > 10;  
        -- skip the current if counter is an even number  
        continue when mod(counter,2) = 1;  
        -- print out the counter  
        raise notice 'counter = %', counter;  
    end loop;  
end; $$
```


How to use a cursor

By default, **SQL statements work with an entire result set** rather than individual rows within a result set.

Occasionally, you may sometimes need to work with the data in a result set one row at a time.

To do that, you can use a **cursor**.

Before you use a cursor to work with individual rows in a result set, you should consider other solutions.

Standard database access is faster and uses fewer server resources than cursor-based access.

The syntax for using a cursor

Declare a cursor

```
DECLARE cursor_name CURSOR FOR select_statement;
```

Declare an error handler for when no rows are found in the cursor

```
DECLARE CONTINUE HANDLER FOR NOT FOUND  
handler_statement;
```

Open the cursor

```
OPEN cursor_name;
```

Get column values from the row and store them in a series of variables

```
FETCH cursor_name INTO variable1[, variable2][,  
variable3]...;
```

Close the cursor

```
CLOSE cursor_name;
```

```
CREATE or replace PROCEDURE sproc.test9()  
language plpgsql as $$  
DECLARE  
    invoice_id_var INT;  
    invoice_total_var DECIMAL(9,2);  
    update_count INT DEFAULT 0;  
    invoices_cursor CURSOR FOR  
        SELECT invoice_id, invoice_total FROM invoices  
        WHERE invoice_total - payment_total - credit_total > 0;  
begin  
    OPEN invoices_cursor;  
    loop  
        FETCH invoices_cursor INTO invoice_id_var, invoice_total_var;  
        exit when not found;  
        IF invoice_total_var > 1000 THEN  
            UPDATE invoices  
                SET credit_total = credit_total + (invoice_total * .1)  
                WHERE invoice_id = invoice_id_var;  
            update_count = update_count + 1;  
        END IF;  
    END loop;  
    CLOSE invoices_cursor;  
    raise notice 'counter = %', CONCAT(update_count, ' row(s) updated.');
```

} Cursor declaration

Fetch next row from the cursor

```
end; $$
```